Generifying your Design Patterns
Part I: The Visitor

by Mark Spritzler

In the upcoming months, I will be writing a series of articles on Generifying your Design Patterns. We will look at a number of Design Patterns from Singleton, Factory, Command, Visitor, and the Template Method. We will see what we can possibly gain from using generics in our design patterns and how to create them. Some patterns work well with generics, and some just don't make sense. Well, you could argue that none of them make sense. After all, all we are accomplishing is an ability to statically type them and have the compiler check that we are using the correct type.

To start off, I have chosen the Visitor pattern, since this is the first one that I attempted to generify. It has worked really well in our application, and although it might put restrictions on the overall usability of a design pattern in certain situations, it is still pretty cool, in my mind. But please note, that this is a hybrid of the Visitor pattern. There is only one visit method in our visitors and are specific to one type. I took this approach, again because I thought it was really cool. But in the next article, in the series, I will go a bit further into how you can implement the true Gang of Four Visitor pattern. It just isn't as cool in my mind as this one.

A generic look at Generics

So let's start by looking at generics and how to make a class generic. I am going to be brief here, since I am hoping that you have a little Generics experience already. We could have lots of articles just explaining generics, but it would be best to assume you have a good understanding of them already and not waste that extra space.

Generics are a way to create a template of your class and allow actual implementations of your template to be type safe. If you were to look at the Javadocs for any of the Collection classes, you would see some interesting characters like E all over the place. The E basically stands for the element type, any type that you will define when you use or extend that class. You can do the same in your classes by just defining the letters that you are going to use.

```
public class MyClass<A, B, C>{}
```

You can use any letters you like. We tend to find that E usually stands for element and T for type. So you will see a lot of people start with T and go alphabetical from there. You usually won't have more than a couple of letters.

Now, in your class code you can use the letters in place of actual Classes. So let's define a method that returns T and takes a U as a parameter.

```
public T myMethod(U parameter)
```

One thing that I have found is that when you return a T or generic, then it is usually either a method in an interface or is declared abstract because you can't create a new T with "new T();". You can call another method that returns a T and assign it to a T reference, but that method that you call will be abstract in your template. (Remember an interface method by default is public and abstract)

Erasure

Another thing to understand is called "erasure". "erasure" means that when javac compiles the code, it will create bytecode that is very much like old Java code that you would have had to create if there wasn't Generics. So what does that mean? It means that the typical Visitor pattern of multiple visit method for each type cannot be generified because the "U parameter" gets "erased" and the compiler wouldn't know

which of your implemented myMethods(accept() method) is actually implementing. Here, let me quote a brilliant mind here.

> We can't really use this interface for visitors with more than one visit method. For example if I try:
>
> ```
> public class PrettyPrintVisitor implements GenerifiedVisitor<OrderHeader>,
>         GenerifiedVisitor<OrderDetail>, GenerifiedVisitor<Part> {
>     public void visit(OrderHeader header) {}
>     public void visit(OrderDetail detail) {}
>     public void visit(Part part) {}
> }
> ```
>
> This is illegal. You can't implement the same generic interface twice with different type arguments. (Since with erasure they'd be exactly the same interface.) To my mind, this severely limits the effectiveness of this generified interface. - *Jim Yingst - JavaRanch Sheriff*.

The Visitor pattern

Now, let's look at the Visitor pattern. In the Gang of Four book, the Visitor Pattern is defined as "Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates."

Here is the basic visitor pattern (remember what I said above, that we are implementing a modified version, but this code here it the real deal):

```
public interface Visitor {
    public void visitA(A o){}
    public void visitB(B o){}
    public void visitC(C o){}
}
```

Any object that accepts a visitor has an accept method:

```
public void accept(Visitor v) {}
```

That was pretty simple, huh?

Now we will generify it:

```
public interface GenericVisitor<T> {
    public void visit(T t){}
}
```

Here is an implementation of that generic visitor:

```
public class MyVisitor implements GenericVisitor<MyClass> {
    public void visit(MyClass myClass) {
    }
}
```

And the object that accepts the visitor:

```
public class MyClass {
     public void accept(GenericVisitor<MyVisitor> visitor) {}
}
```

How about a real example?

Like above we create our GenerifiedVisitor:

```
package com.javaranch.visitor;

public interface GenerifiedVisitor<T> {
    public void visit(T objectToVisit);
}
```

And our GenerifiedVisitable class:

```
package com.javaranch.visitor;

public interface GenerifiedVisitable {
    public <T> void accept(GenerifiedVisitor<T> visitor);
}
```

An example

Looks exactly like what we already wrote, so we should really see it in action. We will create a simple Order Entry program that we need to validate certain pieces of data within an Order. The Order consists of an OrderHeader, OrderDetail, and Part object model. An OrderHeader can have multiple OrderDetails, and each OrderDetail is related to a Part. In our validation program we need to make sure that each Order has a correct customer number and at least one OrderDetail. OrderDetail must have a quantity and a Part. The Part must have a correct part number and price. The exact validation code is not in our sample code for the Visitors to save space, but I am sure you can see how you can create a Visitor for each part of the validations. So I have created one example of each type of Visitor for each part of the Order model, and that code in the visit method will simply just print the object that has been passed in.

So let's look at the Order object model first. The objects are simple JavaBean/POJO classes with getters and setters, and of course the accept method of the GenerifiedVisitable of that type. So first is the Order header. Here is its code:

```
package com.javaranch.visitor.impl.domain;

import com.javaranch.visitor.GenerifiedVisitable;
import com.javaranch.visitor.GenerifiedVisitor;

import java.util.List;
import java.util.ArrayList;

public class OrderHeader implements
    GenerifiedVisitable<OrderHeader> {
    private Long orderId;
    private Long customerId;
    private List<OrderDetail> orderDetails =
        new ArrayList<OrderDetail>();
    private List<ValidationError> errors =
        new ArrayList<ValidationError>();

    // All the Getters and Setters removed for space purposes

    public void accept(GenerifiedVisitor<OrderHeader> visitor) {
        visitor.visit(this);
    }
}
```

As you can see we have four attributes. They have their getters and setters and the accept method which takes only a GenerifiedVisitor that visits OrderHeader objects. Any other type of GenerifiedVisitor that does not visit an OrderHeader object will cause a compile time error. This makes sure you send the right kind of visitor to a visitable class. So what do we gain, what does this save, and what extra code did we have to create? Well, we have gained type safety, but we have added extra code to write by adding the type in those alligator mouths < and >. I think, in order for us to see what else we have saved; we need to look at what code we would have had to write in our visitor classes to understand some of the savings we have created.

Let's say we have a bunch of visitors for all the three objects in our model. Each visitor would need to include a bunch of if statements with `instanceof` to determine if the object passed are of the type that we want to visit. We don't want to have `instanceof` if statements and then have to cast to the correct type. This leaves the checking to be done at run time where it might take more time to find bugs, whereas a compile time check might save us that time of debugging. And we know how much more expensive it is to catch bugs later than earlier. Maintenance and debugging usually takes a significant amount of the costs. I read that somewhere, and I don't want to take that extra time to re-find where I found that. Anyway, let's quickly look at the remaining code.

Here are the OrderDetail and Part objects:

```
package com.javaranch.visitor.impl.domain;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.GenerifiedVisitable;

public class OrderDetail
    implements GenerifiedVisitable<OrderDetail> {
    private Long detailId;
    private OrderHeader header;
    private Part part;
    private int quantity;

    // Getter and Setters removed for space purposes

    public void accept(GenerifiedVisitor<OrderDetail> visitor) {
        visitor.visit(this);
    }
}
```

```
package com.javaranch.visitor.impl.domain;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.GenerifiedVisitable;

public class Part
    implements GenerifiedVisitable<Part>{
    private Long partId;
    private String description;
    private Double price;

    // Getters and Setters removed for space purposes

    public void accept(GenerifiedVisitor<Part> visitor) {
        visitor.visit(this);
    }
}
```

And finally three simple quick visitors that we created:

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.impl.domain.OrderHeader;

public class HeaderVisitor<T>
    implements GenerifiedVisitor<OrderHeader>{
    public void visit(OrderHeader header) {
        System.out.println(header);
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.impl.domain.OrderDetail;

public class DetailVisitor
    implements GenerifiedVisitor<OrderDetail>{
    public void visit(OrderDetail detail) {
        System.out.println(detail);
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.impl.domain.Part;

public class PartVisitor implements GenerifiedVisitor<Part>{
    public void visit(Part part) {
        System.out.println(part);
    }
}
```

Those all look the same, don't they? Well, each Visitor of course would have their own unique code based on what validation the visitor needed to do. Let's create an actual service to create an Order and our Visitors that implement the requirements. We also need to create a new class called ValidationError. This will hold a description of the validation failure that occurs and add it to a new List that we will add to the OrderHeader. When we have run through all our validations, the OrderHeader will have a complete list of all the failed validations and we can print them out. I won't post the code for the ValidationError, but it has one attribute called description and the usual getter and setter for that attribute.

So here are the Visitors:

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;
import com.javaranch.visitor.GenerifiedVisitor;

import java.util.List;

public class CustomerNumberValidationVisitor
    implements GenerifiedVisitor<OrderHeader>{
    public void visit(OrderHeader header) {
        Long customerNumber = header.getCustomerID();
        if (customerNumber == null || customerNumber == 0L) {
            ValidationError error = new ValidationError();
            List<ValidationError> errors =
                header.getValidationErrors();
            error.setErrorDescription("Invalid Customer Number");
            errors.add(error);
        }
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;
import com.javaranch.visitor.impl.domain.OrderDetail;
import com.javaranch.visitor.GenerifiedVisitor;
```

```
import java.util.List;

public class OrderHasDetailValidationVisitor
    implements GenerifiedVisitor<OrderHeader>{
    public void visit(OrderHeader header) {
        List<OrderDetail> details = header.getOrderDetails();
        if (details == null || details.size() == 0) {
            List<ValidationError> errors = header.getValidationErrors();
            errors.add(new ValidationError("There are no Order Lines"););
        }
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.impl.domain.OrderDetail;
import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;

import java.util.List;

public class QuantityValidationVisitor
    implements GenerifiedVisitor<OrderDetail>{
    public void visit(OrderDetail detail) {
        int quantity = detail.getQuantity();
        if (quantity == 0) {
            OrderHeader header = detail().getOrderHeader();
            List<ValidationError> errors = header.getValidationErrors();
            errors.add(new ValidationError("Detail line " +
                                           detail.getDetailId() +
                                           " has no quantity"));
        }
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.impl.domain.OrderDetail;
import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;
import com.javaranch.visitor.impl.domain.Part;
import com.javaranch.visitor.GenerifiedVisitor;

import java.util.List;

public class DetailHasPartValidationVisitor
    implements GenerifiedVisitor<OrderDetail>{
    public void visit(OrderDetail detail) {
        Part part = detail.getPart();
        if (part == null) {
            OrderHeader header = detail.getOrderHeader();
            List<ValidationError> errors = header.getValidationErrors();
            errors.add(new ValidationError("Detail line " +
                                           detail.getDetailId() +
                                           " has no part"));
        }
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.GenerifiedVisitor;
import com.javaranch.visitor.impl.domain.Part;
import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;
```

```
import java.util.List;

public class PartNumberValidationVisitor
    implements GenerifiedVisitor<Part>{
    public void visit(Part part) {
        Long partNumber = part.getPartId();
        if (partNumber == null || partNumber == 0) {
            OrderHeader header = part.getDetail().getOrderHeader();
            List<ValidationError> errors = header.getValidationErrors();
            errors.add(new ValidationError("Invalid Part Number"));
        }
    }
}
```

```
package com.javaranch.visitor.impl.visitors;

import com.javaranch.visitor.impl.domain.Part;
import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.ValidationError;
import com.javaranch.visitor.GenerifiedVisitor;

import java.util.List;

public class PriceValidationVisitor
    implements GenerifiedVisitor<Part>{
    public void visit(Part part) {
        Double price = part.getPrice();
        if (price == null || price == 0) {
            OrderHeader header = part.getDetail().getOrderHeader();
            List<ValidationError> errors = header.getValidationErrors();
            errors.add(new ValidationError("Price for part:
                                    " + part.getPartId() +
                                    " is invalid"));
        }
    }
}
```

In all cases, I need to get the OrderHeader, if I don't have it already, when a validation fails, so that I can add a new ValidationError to its error list.

So, now, really finally, here is the actual service class that will use these Validations and validate an Order:

```
package com.javaranch.visitor;

import com.javaranch.visitor.impl.domain.OrderHeader;
import com.javaranch.visitor.impl.domain.OrderDetail;
import com.javaranch.visitor.impl.domain.Part;
import com.javaranch.visitor.impl.domain.ValidationError;
import com.javaranch.visitor.impl.visitors.CustomerNumberValidationVisitor;
import com.javaranch.visitor.impl.visitors.OrderHasDetailValidationVisitor;
import com.javaranch.visitor.impl.visitors.QuantityValidationVisitor;
import com.javaranch.visitor.impl.visitors.DetailHasPartValidationVisitor;
import com.javaranch.visitor.impl.visitors.PartNumberValidationVisitor;
import com.javaranch.visitor.impl.visitors.PriceValidationVisitor;

import java.util.List;

public class VisitorService {
    public void validate(OrderHeader header) {
        CustomerNumberValidationVisitor visitor1 =
            new CustomerNumberValidationVisitor();
        OrderHasDetailValidationVisitor visitor2 =
            new OrderHasDetailValidationVisitor();
        QuantityValidationVisitor visitor3       =
            new QuantityValidationVisitor();
        DetailHasPartValidationVisitor visitor4  =
```

```
            new DetailHasPartValidationVisitor();
        PartNumberValidationVisitor visitor5     =
            new PartNumberValidationVisitor();
        PriceValidationVisitor visitor6          =
            new PriceValidationVisitor();

        header.accept(visitor1);
        header.accept(visitor2);
        List<OrderDetail> details = header.getOrderDetails();
        for (OrderDetail detail: details) {
            detail.accept(visitor3);
            detail.accept(visitor4);
            Part part = detail.getPart();
            part.accept(visitor5);
            part.accept(visitor6);
        }

        for (ValidationError error : header.getValidationErrors()) {
            System.out.println("Error! " + error.getErrorDescription());
        }

    }
}
```

Conclusion

Wasn't that fun? I think the capability of Generics to increase type safety is a good thing. Some people don't want that strict static typing. In many cases, you will have to code a little more, and in many other cases, you will find that you save a lot of coding. Some design patterns really work well with generics and others don't. It really is a give and take, trial and error kind of process, but one that is really fun, at least to me. My favorite design pattern is the Template Method pattern, and I have found it to work extremely well with generics. I have implemented a cache factory of lookups to store values needed for ComboBoxes on a client. Using generics and the Template Method, I have made this extremely easy to create lookups, as well as an unexpected pleasant surprise when creating the ComboBoxes' data model. So look to the next issue of the JavaRanch newsletter for the second in a series of articles on "Generifying your Design Patterns". I hope you enjoyed this article as much as I did.